

Automated reconciliation: If someone accidentally changes something in the cluster, GitOps corrects it.

Auditability and rollbacks: Git's version history provides transparency and easy rollback options.

Think of GitOps like a thermostat. You set your desired temperature (your infrastructure state) and the system keeps adjusting until it matches your configuration. If the room gets too cold, the heat kicks in; if something in your cluster diverges, GitOps agents bring it back in line.

GitOps in action: Typical workflow

Here's a basic workflow to help visualise how GitOps fits into DevOps teams.

Code and config commit: Devs push application code and YAML manifests (e.g., for services, deployments) to Git.

GitOps agent detects change: FluxCD or ArgoCD watches the repo and notices the update.

Sync to cluster: The agent syncs changes to the Kubernetes cluster, creating or updating resources.

Monitoring and reconciliation: If something drifts from the declared state, the tool re-aligns the system.

This hands-off approach not only saves time but also increases reliability and reduces errors.

Setting up GitOps: Step-by-step walkthrough

Let's make this actionable with a beginner-friendly setup outline.

- **Prepare Git repositories**
 - Separate repos for infrastructure and app configuration (infra.git, app.git).
 - Use YAML manifests for Kubernetes resources.
- **Install FluxCD or ArgoCD**
 - Use Helm charts, kubectl, or CLI tools.
 - Configure the Git source and target clusters.
- **Define sync rules**
 - Decide how and when to sync (e.g., automatically on commit or via approval).
- **Add secrets and permissions**
 - Use SOPS, Vault, or Sealed Secrets for sensitive data.
 - Set up RBAC for team roles.
- **Test and monitor**
 - Deploy a test app, observe health, and check rollback behaviour.

Real world scenario: A QA automation engineer defines test environment configs in Git. Each update triggers automated cluster updates, ensuring consistency across test cycles and reducing manual provisioning.

FluxCD and ArgoCD: Feature comparison

The following table dives into the two powerhouses of GitOps: FluxCD and ArgoCD.

Feature	FluxCD	ArgoCD
Sync mechanism	Automatic, continuous pull-based	Manual or automatic sync options
User interface	CLI-heavy, YAML-centric	Rich UI and dashboards
Secrets management	SOPS, native Kubernetes secrets	Vault, SealedSecrets, external tools
Multi-cluster support	Yes, via repo segmentation	Yes, centralised control panel
Alerting and monitoring	Prometheus/Grafana integration	Built-in dashboards + third-party support
Rollbacks	Git-based rollback	One-click rollback with preview options
GitOps philosophy	Lightweight, Kubernetes-native	Platform-centric, broader scope

FluxCD: A developer's minimalist tool

Key strengths of FluxCD are:

- Seamlessly fits into Kubernetes-native workflows
- Ideal for small teams or microservice deployments
- Lightweight and fast with fewer moving parts

Case study: A startup with three developers uses FluxCD to manage their staging and production clusters. With Git as the sole source of truth, deployment errors have dropped drastically, and onboarding has become easier.

Features of FluxCD you'll love are:

- Git commit triggers syncs — no UI approval required
- Compatible with Helm, Kustomize, and Terraform
- Integrates with observability tools using native eventing

ArgoCD: A platform engineer's favourite

Here are ArgoCD's key strengths:

- Rich UI with dashboards showing health, history, and rollout status
- Scales easily across environments with user-based access
- Supports custom sync policies and hooks

Case study: A financial services company with 100+ apps uses ArgoCD for production deployments. RBAC controls ensure only leads can trigger changes, while dashboards provide transparency during audits.

Features that stand out in ArgoCD are:

- Side-by-side diff viewer before applying changes
- Easy rollback interface for production stability
- Sync locks to restrict deployments outside working hours

Choosing between FluxCD and ArgoCD

Selecting between FluxCD and ArgoCD isn't a question of which tool is "better" — it's about understanding your team's